

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

on

OPERATING SYSTEMS

Submitted by

AISHWARYA M (1BM24CS024)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by AISHWARYA M(1BM24CS024), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026 . The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Faculty Incharge Name
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1a.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. →FCFS → SJF (pre-emptive & Non-preemptive)	1-8
1b.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. → Priority (pre-emptive & Non-pre-emptive) →Round Robin (Experiment with different quantum sizes for RR algorithm)	9-16
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	17-20
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	21-27
4.	Write a C program to simulate: a) Producer-consumer problem using semaphores b) Dining-Philosopher's problem	28-35
5.	Write a C program to simulate a) Banker's algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	36-41
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	42-45
7	Write a C program to simulate page replacement algorithms a) FIFO b) LRU c) Optimal	46-51

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program 1a)

Question: Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

Code: =>FCFS:

```
#include<stdio.h>

struct Process{
    int pid;
    int arrival;
    int burst;
    int completion;
    int waiting;
    int turnaround;
};

int main(){
    int n,i,j;
    struct Process p[100],temp;

    printf("enter number of processes\n");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("\n");
        printf("P%d\n",i+1);
        p[i].pid=i+1;
        printf("enter arrival\n");
        scanf("%d",&p[i].arrival);
        printf("enter burst\n");
        scanf("%d",&p[i].burst);
    }
```

```

for(i=0;i<n;i++){
    for(j=0;j<n-i-1;j++){
        if(p[j].arrival > p[j+1].arrival){
            temp=p[j];
            p[j]=p[j+1];
            p[j+1]=temp;
        }
    }
}

int current=0;
for(i=0;i<n;i++){
    if(current<p[i].arrival){
        current=p[i].arrival;
    }
    p[i].completion=current+p[i].burst;
    p[i].turnaround=p[i].completion-p[i].arrival;
    p[i].waiting=p[i].turnaround-p[i].burst;
    current =p[i].completion;
}

float totalWT = 0, totalTAT = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].arrival,
        p[i].burst,

```

p[i].completion,

p[i].turnaround,

p[i].waiting);

totalWT += p[i].waiting;

totalTAT += p[i].turnaround;

}

printf("\naverage waiting time = %.2f", totalWT / n);

printf("\naverage turnaround time = %.2f\n", totalTAT / n);

return 0;

}

Result:

The screenshot shows a C++ IDE with the following code for the FCFS scheduling algorithm:

```
1 #include<stdio.h>
2 struct Process{
3     int pid;
4     int arrival;
5     int burst;
6     int completion;
7     int waiting;
8     int turnaround;
9 };
10 int main(){
11     int n,i,j;
12     struct Process p[100],temp;
13
14     printf("enter number of processes\n");
15     scanf("%d",&n);
16     for(i=0;i<n;i++){
17         printf("\n");
18         printf("P%d\n",i+1);
19         p[i].pid=i+1;
20         printf("enter arrival\n");
21         scanf("%d",&p[i].arrival);
22         printf("enter burst\n");
23         scanf("%d",&p[i].burst);
24     }
25
26     for(i=0;i<n;i++){
27         for(j=0;j<n-i-1;j++){
28             if(p[j].arrival > p[j+1].arrival){
29                 temp=p[j];
30                 p[j]=p[j+1];
31                 p[j+1]=temp;
32             }
33         }
34     }
35 }
```

The output of the program is as follows:

```
enter number of processes
4
P1
enter arrival
0
enter burst
2
P2
enter arrival
1
enter burst
2
P3
enter arrival
5
enter burst
3
P4
enter arrival
6
enter burst
4
PID  AT  BT  CT  TAT  WT
1    0   2   2   2   0
2    1   2   4   3   1
3    5   3   8   3   0
4    6   4  12   6   2

average waiting time = 0.75
average turnaround time = 3.50

Process returned 0 (0x0)   execution time : 26.210 s
Press any key to continue.
```

Code: => SJF(Non-preemptive):

```
#include <stdio.h>

struct Process{
    int pid;
    int at;
    int bt;
    int ct;
    int tat;
    int wt;
    int completed;
};

int main(){
    struct Process p[20];
    int n,i,time=0,completed=0,min_bt,index;
    float avg_wt=0,avg_tat=0;
    printf("enter no of process: \n");
    scanf("%d",&n);

    for(i=0;i<n;i++){
        printf("\n");
        printf("P%d\n",i+1);
        p[i].pid=i+1;
        printf("enter arrival time \n");
        scanf("%d",&p[i].at);
        printf("enter burst time: \n");
        scanf("%d",&p[i].bt);
        p[i].completed=0;
    }

    while(completed != n){
        min_bt=9999;
```

```

index=-1;
for(i=0;i<n;i++){
    if(p[i].at <= time && p[i].completed == 0 && p[i].bt < min_bt){
        min_bt = p[i].bt;
        index=i;
    }
}

if(index != -1){
    time += p[index].bt;
    p[index].ct=time;
    p[index].tat=p[index].ct-p[index].at;
    p[index].wt=p[index].tat-p[index].bt;

    avg_wt += p[index].wt;
    avg_tat += p[index].tat;

    p[index].completed =1;
    completed++;
}else{
    time++;
}
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
}

printf("\naverage turnaround time = %.2f\n",avg_tat/n);
printf("\naverage waiting time = %.2f\n",avg_wt/n);
return 0;
}

```


Result:

```
#include <stdio.h>

struct Process{
    int pid;
    int at;
    int bt;
    int rt;
    int ct;
    int tat;
    int wt;
    int completed;
};

int main(){
    struct Process p[20];
    int n,i,time=0,completed=0,min_bt,index;
    float avg_wt=0,avg_tat=0;
    printf("enter no of process: \n");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("\n");
        printf("P%d\n",i+1);
        p[i].pid=i+1;
        printf("enter arrival time \n");
        scanf("%d",&p[i].at);
        printf("enter burst time: \n");
        scanf("%d",&p[i].bt);
        p[i].completed=0;
    }
    while(completed != n){
        min_bt=9999;
        index=-1;
        for(i=0;i<n;i++){
            if(p[i].at <= time && p[i].completed == 0 && p[i].bt < min_bt){
                min_bt = p[i].bt;
                index=i;
            }
        }
        if(index != -1){
            time += p[index].bt;
            p[index].ct=time;
            p[index].tat=p[index].ct-p[index].at;
            p[index].wt=p[index].tat-p[index].bt;
        }
        p[index].completed++;
    }
    for(i=0;i<n;i++){
        avg_wt += p[i].wt;
        avg_tat += p[i].tat;
    }
    avg_wt /= n;
    avg_tat /= n;
    printf("average turnaround time = %.2f\n",avg_tat);
    printf("average waiting time = %.2f\n",avg_wt);
    printf("Process returned 0 (0x0)   execution time : 14.719 s\n");
    printf("Press any key to continue.\n");
    getch();
}
```

enter no of process:
4
P1
enter arrival time
0
enter burst time:
7
P2
enter arrival time
8
enter burst time:
3
P3
enter arrival time
3
enter burst time:
4
P4
enter arrival time
5
enter burst time:
6

PID	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	8	3	14	6	3
P3	3	4	11	8	4
P4	5	6	20	15	9

average turnaround time = 9.00
average waiting time = 4.00
Process returned 0 (0x0) execution time : 14.719 s
Press any key to continue.

Code: ==>SJF(preemptive):

```
#include <stdio.h>
```

```
struct Process{
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int rt;
```

```
    int ct;
```

```
    int tat;
```

```
    int wt;
```

```
};
```

```
int main(){
```

```
    struct Process p[20];
```

```
    int n,i,time=0,completed=0,shortest,min_rt;
```

```

float avg_wt=0,avg_tat=0;
printf("enter no of process: \n");
scanf("%d",&n);
for(i=0;i<n;i++){
    printf("\n");
    printf("P%d\n",i+1);
    p[i].pid=i+1;
    printf("enter arrival time \n");
    scanf("%d",&p[i].at);
    printf("enter burst time: \n");
    scanf("%d",&p[i].bt);
    p[i].rt=p[i].bt;
}
while(completed != n){
    shortest=-1;
    min_rt=9999;
    for(i=0;i<n;i++){
        if(p[i].at <= time && p[i].rt>0 && p[i].rt<min_rt){
            min_rt=p[i].rt;
            shortest=i;
        }
    }
    if(shortest == -1){
        time++;
        continue;
    }
    p[shortest].rt--;
    time++;
    if(p[shortest].rt == 0){
        completed++;
    }
}

```

```

    p[shortest].ct=time;
    p[shortest].tat=p[shortest].ct-p[shortest].at;
    p[shortest].wt=p[shortest].tat-p[shortest].bt;
    avg_tat+=p[shortest].tat;
    avg_wt += p[shortest].wt;
}
}
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
}
printf("\naverage turnaround time = %.2f\n",avg_tat/n);
printf("\naverage waiting time = %.2f\n",avg_wt/n);
return 0;
}

```

Result:

The screenshot shows the Code::Blocks IDE with a C++ project named 'gjf_premp.c'. The source code is visible in the left pane, and the output of the program is shown in the right pane.

Source Code (gjf_premp.c):

```

1 #include <stdio.h>
2 struct Process{
3     int pid;
4     int at;
5     int bt;
6     int rt;
7     int ct;
8     int tat;
9     int wt;
10 };
11 int main(){
12     struct Process p[20];
13     int n,i,time=0,completed=0,shortest,min_rt;
14     float avg_wt=0,avg_tat=0;
15     printf("Enter no of process: \n");
16     scanf("%d",&n);
17     for(i=0;i<n;i++){
18         printf("P%d\n",i+1);
19         printf("Enter arrival time \n");
20         scanf("%d",&p[i].at);
21         printf("Enter burst time: \n");
22         scanf("%d",&p[i].bt);
23         p[i].rt=p[i].bt;
24     }
25     while(completed != n){
26         shortest=-1;
27         for(i=0;i<n;i++){
28             if(p[i].at <= time && p[i].rt>0 && p[i].rt<min_rt){
29                 min_rt=p[i].rt;
30                 shortest=i;
31             }
32         }
33         if(shortest == -1){
34             time++;
35             continue;
36         }
37         p[shortest].rt--;
38         p[shortest].ct=time;
39         p[shortest].tat=p[shortest].ct-p[shortest].at;
40         p[shortest].wt=p[shortest].tat-p[shortest].bt;
41         avg_tat+=p[shortest].tat;
42         avg_wt+=p[shortest].wt;
43         completed++;
44     }
45     printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
46     for(i=0;i<n;i++){
47         printf("P%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
48     }
49     printf("\naverage turnaround time = %.2f\n",avg_tat/n);
50     printf("\naverage waiting time = %.2f\n",avg_wt/n);
51     return 0;
52 }

```

Output (Aishwarya MvjL_premp.exe):

```

enter no of process:
4
P1
enter arrival time
0
enter burst time:
8
P2
enter arrival time
1
enter burst time:
4
P3
enter arrival time
2
enter burst time:
9
P4
enter arrival time
3
enter burst time:
5

PID  AT  BT  CT  TAT  WT
P1   0   8   17  17   9
P2   1   4   5   4   0
P3   2   9   26  24  15
P4   3   5   10  7   2

average turnaround time = 13.00
average waiting time = 6.50

Process returned 0 (0x0)   execution time : 52.261 s
Press any key to continue.

```

Program 1b)

Question: Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

→ Priority (pre-emptive & Non-pre-emptive)

→ Round Robin (Experiment with different quantum sizes for RR algorithm)

Code : → Priority (Non-pre-emptive)

```
#include<stdio.h>

struct Process{
    int pid;
    int at, bt, pr;
    int ct, tat, wt;
    int done;
};

int main(){
    int n, i, time=0, completed=0, idx;
    float avg_tat=0, avg_wt=0;
    struct Process p[20];
    printf("enter number of process: \n");
    scanf("%d", &n);
    for(i=0; i<n; i++){
        p[i].pid=i+1;
        printf("\nP%d\n", p[i].pid);
        printf("enter at bt and priority : \n");
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);

        p[i].done=0;
    }
    while(completed < n){
        idx=-1;
        for(i=0; i<n; i++){
            if(p[i].at <= time && p[i].done == 0){
                if(idx == -1 || p[i].pr < p[idx].pr){
                    idx=i;
                }
            }
        }
        if(idx != -1){
            p[idx].done=1;
            completed++;
            time += p[idx].bt;
            avg_tat += p[idx].tat;
            avg_wt += p[idx].wt;
        }
    }
    printf("Average TAT: %f\n", avg_tat/n);
    printf("Average WT: %f\n", avg_wt/n);
}
```

```

        }
    }
}
if(idx == -1){
    time++;
}else{
    time += p[idx].bt;
    p[idx].ct=time;
    p[idx].tat=p[idx].ct-p[idx].at;
    p[idx].wt=p[idx].tat - p[idx].bt;
    p[idx].done=1;
    avg_tat += p[idx].tat;
    avg_wt += p[idx].wt;
    completed++;

}
}
printf("\nPID\tAT\tBT\tPRI\tCT\tTAT\tWT\n");
for(i=0;i<n;i++){
    printf("\n%d\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].pr,p[i].ct,p[i].tat,p[i].wt);
}
printf("\n");
printf("Average Turnaround time : %.2f ",avg_tat/n);
printf("Average waiting time : %.2f ",avg_wt/n);
return 0;
}

```

Result:

The screenshot shows a C++ IDE with a file named `priority_non_pree.c`. The code implements a priority scheduling algorithm. The output window shows the following:

```

enter number of process:
5
P1
enter at bt and priority :
0 3 5
P2
enter at bt and priority :
2 2 3
P3
enter at bt and priority :
3 5 2
P4
enter at bt and priority :
4 4 4
P5
enter at bt and priority :
6 1 1

PID  AT   BT   PRI   CT   TAT   WT
1    0    3    5    3    3    0
2    2    2    3   11    9    7
3    3    5    2    8    5    0
4    4    4    4   15   11    7
5    6    1    1    9    3    2

Average Turnaround time : 6.20 Average waiting time : 3.20
Process returned 0 (0x0)   execution time : 29.135 s
Press any key to continue.

```

Code : → Priority (pre-emptive)

```
#include<stdio.h>
```

```
struct Process{
```

```
    int pid;
```

```
    int at, bt, pr;
```

```
    int rt;
```

```
    int ct, tat, wt;
```

```
};
```

```
int main(){
```

```
    int n, i, time=0, completed=0, idx;
```

```
    float avg_tat=0, avg_wt=0;
```

```
    struct Process p[20];
```

```
    printf("enter number of process: \n");
```

```
    scanf("%d", &n);
```

```
    for(i=0; i<n; i++){
```

```
        p[i].pid=i+1;
```

```
        printf("\nP%d\n", p[i].pid);
```

```
        printf("enter at bt and priority : \n");
```

```

scanf("%d %d %d",&p[i].at,&p[i].bt,&p[i].pr);

p[i].rt=p[i].bt;
}
while(completed < n){
    idx=-1;
    for(i=0;i<n;i++){
        if(p[i].at <= time && p[i].rt>0){
            if(idx == -1 || p[i].pr < p[idx].pr){
                idx=i;
            }
        }
    }
    if(idx == -1){
        time++;
    }else{
        p[idx].rt--;
        time ++;
        if(p[idx].rt == 0){
            p[idx].ct=time;
            p[idx].tat=p[idx].ct-p[idx].at;
            p[idx].wt=p[idx].tat - p[idx].bt;

            avg_tat += p[idx].tat;
            avg_wt += p[idx].wt;
            completed++;
        }
    }
}
printf("\nPID\tAT\tBT\tPRI\tCT\tTAT\tWT\n");
for(i=0;i<n;i++){
    printf("\n%d\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].pr,p[i].ct,p[i].tat,p[i].wt);
}
printf("\n");

```

```

printf("Average Turnaround time : %f",avg_tat/n);
printf("Average waiting time : %f",avg_wt/n);

return 0;
}

```

Result:

The screenshot shows the Code::Blocks IDE with a C++ project named 'priority_preec'. The source code is visible on the left, and the output window on the right displays the program's execution results.

Source Code (priority_preec.c):

```

1 #include<stdio.h>
2 struct Process{
3     int pid;
4     int at,bt,pr;
5     int rt;
6     int ct,tat,wt;
7 };
8
9 int main(){
10     int n,i,time=0,completed=0,idx;
11     float avg_tat=0,avg_wt=0;
12     struct Process p[20];
13     printf("Enter number of process: \n");
14     scanf("%d",&n);
15     for(i=0;i<n;i++){
16         p[i].pid=i+1;
17         printf("Enter at and priority : \n");
18         scanf("%d %d",&p[i].at,&p[i].bt,&p[i].pr);
19     }
20     p[i].rt=p[i].bt;
21     while(completed < n){
22         idx=-1;
23         for(i=0;i<n;i++){
24             if(p[i].at <= time && p[i].rt>0){
25                 if(idx == -1 || p[i].pr < p[idx].pr){
26                     idx=i;
27                 }
28             }
29         }
30         if(idx == -1){
31             time++;
32         }else{
33             p[idx].rt--;
34             time++;
35             if(p[idx].rt == 0){
36                 p[idx].ct=time;
37                 p[idx].tat=p[idx].ct-p[idx].at;
38                 p[idx].wt=p[idx].tat - p[idx].bt;
39             }
40         }
41     }
42     avg_tat=(p[0].tat+p[1].tat+...+p[n-1].tat)/n;
43     avg_wt=(p[0].wt+p[1].wt+...+p[n-1].wt)/n;
44     printf("Average Turnaround time : %f\n",avg_tat);
45     printf("Average waiting time : %f\n",avg_wt);
46     return 0;
47 }

```

Output Window:

```

enter number of process:
5
P1
enter at bt and priority :
0 3 3
P2
enter at bt and priority :
1 4 2
P3
enter at bt and priority :
2 6 4
P4
enter at bt and priority :
3 4 6
P5
enter at bt and priority :
5 2 10

```

PID	AT	BT	PRI	CT	TAT	WT
1	0	3	3	7	7	4
2	1	4	2	5	4	0
3	2	6	4	13	11	5
4	3	4	6	17	14	10
5	5	2	10	19	14	12

Average Turnaround time : 10.800000 Average waiting time : 6.200000
Process returned 0 (0x0) execution time : 65.439 s
Press any key to continue.

Code : → Round Robin

```
#include <stdio.h>
```

```

struct Process {
    int pid;
    int at;
    int bt;
    int rt;
    int ct;
    int tat;
    int wt;
}

```



```

    int exist;
};

int main() {
    int n, tq, t = 0, completed = 0;
    struct Process p[10];
    int queue[100], front = 0, rear = 0;
    float avg_tat = 0, avg_wt = 0;

    printf("Enter Total Process: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter Arrival and Burst Time for P%d: ", i + 1);
        scanf("%d %d", &p[i].at, &p[i].bt);
        p[i].pid = i + 1;
        p[i].rt = p[i].bt;
        p[i].exist = 0;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    for (int i = 0; i < n; i++) {
        if (p[i].at <= t) {
            queue[rear++] = i;
            p[i].exist = 1;
        }
    }

    while (front < rear) {

        int i = queue[front++];

        if (p[i].rt > tq) {

```

```

    t += tq;
    p[i].rt -= tq;
}
else {
    t += p[i].rt;
    p[i].rt = 0;
    completed++;

    p[i].ct = t;
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
}

for (int j = 0; j < n; j++) {
    if (p[j].at <= t && p[j].exist == 0) {
        queue[rear++] = j;
        p[j].exist = 1;
    }
}
if (p[i].rt > 0) {
    queue[rear++] = i;
}
if (front == rear && completed < n) {
    for (int j = 0; j < n; j++) {
        if (p[j].exist == 0) {
            t = p[j].at;
            queue[rear++] = j;
            p[j].exist = 1;
            break;
        }
    }
}
}
}

```

```

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt,
        p[i].ct, p[i].tat, p[i].wt);

    avg_tat += p[i].tat;
    avg_wt += p[i].wt;
}
avg_tat /= n;
avg_wt /= n;
printf("\nAverage Turnaround Time = %.2f", avg_tat);
printf("\nAverage Waiting Time = %.2f", avg_wt);

return 0;
}

```

Result:

The screenshot shows the Code::Blocks IDE with the file `round_robin.c` open. The code implements a round-robin scheduling algorithm. The output window shows the following results:

```

Enter Total Process: 5
Enter Arrival and Burst Time for P1: 0 5
Enter Arrival and Burst Time for P2: 1 3
Enter Arrival and Burst Time for P3: 2 1
Enter Arrival and Burst Time for P4: 3 2
Enter Arrival and Burst Time for P5: 4 3
Enter Time Quantum: 2

Process AT    BT    CT    TAT    WT
P1      0      5     13     13     8
P2      1      3     12     11     8
P3      2      1      5      3      2
P4      3      2      9      6      4
P5      4      3     14     10     7

Average Turnaround Time = 8.60
Average Waiting Time = 5.80
Process returned 0 (0x0)   execution time : 18.747 s
Press any key to continue.

```

The bottom of the screenshot shows the 'Logs & others' panel with the following message:

```

=== Build file: "no target" in "no project" (compiler: unknown) ===
=== Build finished: 0 error(s), 0 warning(s) (0 minute(s), 0 second(s)) ===

```

Program 2)

Question: Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

```
#include <stdio.h>

#define MAX 20

struct Process {
    int pid, at, bt, rt;
    int ct, tat, wt;
    int qno;
};

int main() {
    int n;
    struct Process p[MAX];
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter AT BT QueueNo(1-RR,2-FCFS):\n");
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].qno);
        p[i].rt = p[i].bt;
    }
    int time = 0, completed = 0;
    int tq = 2;

    while (completed < n) {
        int executed = 0;
        for (int i = 0; i < n; i++) {

            if (p[i].qno == 1 &&
```

```

    p[i].rt > 0 &&
    p[i].at <= time) {

    executed = 1;
    int run = (p[i].rt > tq) ? tq : p[i].rt;

    time += run;
    p[i].rt -= run;

    if (p[i].rt == 0) {
        p[i].ct = time;
        completed++;
    }
}

if (executed) continue;
int idx = -1;
int earliest = 99999;

for (int i = 0; i < n; i++) {
    if (p[i].qno == 2 &&
        p[i].rt > 0 &&
        p[i].at <= time) {

        if (p[i].at < earliest) {
            earliest = p[i].at;
            idx = i;
        }
    }
}

if (idx != -1) {

    executed = 1;

```

```

while (p[idx].rt > 0) {
    int q1_ready = 0;

    for (int j = 0; j < n; j++) {
        if (p[j].qno == 1 &&
            p[j].rt > 0 &&
            p[j].at <= time) {
            q1_ready = 1;
            break;
        }
    }

    if (q1_ready)
        break;
    time++;
    p[idx].rt--;
}

if (p[idx].rt == 0) {
    p[idx].ct = time;
    completed++;
}
}

if (!executed)
    time++;
}

float avg_wt = 0, avg_tat = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for (int i = 0; i < n; i++) {
    p[i].tat = p[i].ct - p[i].at;

```

```

p[i].wt = p[i].tat - p[i].bt;
printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
    p[i].pid,
    p[i].at,
    p[i].bt,
    p[i].ct,
    p[i].tat,
    p[i].wt);
avg_wt += p[i].wt;
avg_tat += p[i].tat;
}
printf("\nAvg WT = %.2f", avg_wt / n);
printf("\nAvg TAT = %.2f\n", avg_tat / n);
return 0;
}

```

Result:

The screenshot displays the Code::Blocks IDE with the following components:

- Source Code (multi_queue.c):**

```

1 #include <stdio.h>
2
3 #define MAX 20
4
5 struct Process {
6     int pid, at, bt, rt;
7     int ct, tat, wt;
8     int qno;
9 };
10
11 int main() {
12
13     int n;
14     struct Process p[MAX];
15
16     printf("Enter number of processes:");
17     scanf("%d", &n);
18
19     printf("Enter AT BT QueueNo(1-RR,2-FCFS):");
20
21     for (int i = 0; i < n; i++) {
22         p[i].pid = i + 1;
23         scanf("%d %d %d", &p[i].at, &p[i].bt);
24         p[i].rt = p[i].bt;
25     }
26
27     int time = 0, completed = 0;
28     int tq = 2;
29
30     while (completed < n) {
31
32         int executed = 0;
33
34         for (int i = 0; i < n; i++) {

```
- Output Console:**

```

Enter number of processes: 4
Enter AT BT QueueNo(1-RR,2-FCFS):
0 4 1
0 3 1
0 8 2
10 5 1

PID   AT   BT   CT   TAT  WT
P1    0   4   6   6   2
P2    0   3   7   7   4
P3    0   8  20  20  12
P4   10   5  15   5   0

Avg WT = 4.50
Avg TAT = 9.50

Process returned 0 (0x0)   execution time : 18.276 s
Press any key to continue.

```
- Status Bar:** Shows the file path `\\Users\\Aishwarya M\\Downloads\\OS\\multi_queue.c` and cursor position `C/C++ Windows (CR+LF) WINDOWS-1252 Line 116, Col 24, Pos 2485`.

Program 3)

Question: Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a. Rate- Monotonic
- b. Earliest-deadline First
- c. Proportional scheduling

Code: a. Rate- Monotonic

```
#include <stdio.h>
#include <math.h>

#define MAX 10

int find_lcm(int a, int b) {
    int max = (a > b) ? a : b;
    while (1) {
        if (max % a == 0 && max % b == 0)
            return max;
        max++;
    }
}

int main() {
    int n;
    printf("Enter the number of processes:");
    scanf("%d", &n);

    int C[MAX], P[MAX];

    printf("Enter the CPU burst times:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &C[i]);

    printf("Enter the time periods:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &P[i]);

    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (P[i] > P[j]) {
                int temp;

                temp = P[i]; P[i] = P[j]; P[j] = temp;
                temp = C[i]; C[i] = C[j]; C[j] = temp;
            }
        }
    }
}
```



```

}

int lcm = P[0];
for (int i = 1; i < n; i++) {
    lcm = find_lcm(lcm, P[i]);
}

printf("LCM=%d\n\n", lcm);

printf("Rate Monotone Scheduling:\n");
printf("PID   Burst   Period\n");
for (int i = 0; i < n; i++) {
    printf("%d    %d    %d\n", i + 1, C[i], P[i]);
}

float U = 0;
for (int i = 0; i < n; i++) {
    U += (float)C[i] / P[i];
}

float bound = n * (pow(2, (float)1 / n) - 1);

printf("\n%f <= %f => %s\n", U, bound, (U <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n\n", lcm);

int remaining[MAX] = {0};
int current = -1;

for (int t = 0; t < lcm; t++) {

    for (int i = 0; i < n; i++) {
        if (t % P[i] == 0) {
            remaining[i] = C[i];
        }
    }

    int next = -1;
    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            next = i;
            break;
        }
    }

    if (next != current) {
        if (next == -1)

```

```

        printf("%dms onwards: CPU is idle\n", t);
    else
        printf("%dms onwards: Process %d running\n", t, next + 1);
    current = next;
}

if (next != -1) {
    remaining[next]--;
}
}

return 0;
}

```

Result:

The screenshot shows a code editor with the following C code for Rate Monotone Scheduling:

```

1 #include <stdio.h>
2 #include <math.h>
3
4 #define MAX 10
5
6 int find_lcm(int a, int b) {
7     int max = (a > b) ? a : b;
8     while (1) {
9         if (max % a == 0 && max % b == 0)
10             return max;
11         max++;
12     }
13 }
14
15 int main() {
16     int n;
17     printf("Enter the number of processes:");
18     scanf("%d", &n);
19
20     int C[MAX], P[MAX];
21
22     printf("Enter the CPU burst times:\n");
23     for (int i = 0; i < n; i++)
24         scanf("%d", &C[i]);
25
26     printf("Enter the time periods:\n");
27     for (int i = 0; i < n; i++)
28         scanf("%d", &P[i]);
29
30     for (int i = 0; i < n - 1; i++) {
31         for (int j = i + 1; j < n; j++) {
32             if (P[i] > P[j]) {
33                 int temp;
34                 temp = P[i]; P[i] = P[j]; P[j] = temp;
35                 temp = C[i]; C[i] = C[j]; C[j] = temp;
36             }
37         }
38     }
39
40     int lcm = P[0];
41     for (int i = 1; i < n; i++) {
42         lcm = find_lcm(lcm, P[i]);
43     }
44
45     printf("LCM=%d\n", lcm);
46 }

```

The terminal output shows the execution of the program:

```

Enter the number of processes:3
Enter the CPU burst times:
1 2 3
Enter the time periods:
4 6 12
LCM=12

Rate Monotone Scheduling:
PID  Burst  Period
1      1      4
2      2      6
3      3     12

0.833333 <= 0.779763 => false
Scheduling occurs for 12 ms

0ms onwards: Process 1 running
1ms onwards: Process 2 running
3ms onwards: Process 3 running
4ms onwards: Process 1 running
5ms onwards: Process 3 running
6ms onwards: Process 2 running
8ms onwards: Process 1 running
9ms onwards: Process 3 running
10ms onwards: CPU is idle

Process returned 0 (0x0)   execution time : 10.626 s
Press any key to continue.

```

Code: b. Earliest-deadline First

```

#include <stdio.h>

#define MAX 10

int main() {
    int n;

    printf("Enter number of processes: ");

```

```

scanf("%d", &n);

int burst[MAX], deadline[MAX], period[MAX];
int remaining[MAX], abs_deadline[MAX];

printf("Enter Burst Time, Initial Deadline, Period:\n");
for (int i = 0; i < n; i++) {
    scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);
    remaining[i] = burst[i];
    abs_deadline[i] = deadline[i];
}

printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\n", i+1, burst[i], deadline[i], period[i]);
}

int time = 0, end_time;

printf("\nEnter total scheduling time: ");
scanf("%d", &end_time);

printf("\nScheduling occurs for %d ms\n\n", end_time);

while (time < end_time) {

    for (int i = 0; i < n; i++) {
        if (time != 0 && time % period[i] == 0) {
            remaining[i] = burst[i];
            abs_deadline[i] = time + deadline[i];
        }
    }

    int selected = -1;
    int min_deadline = 9999;

    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            if (abs_deadline[i] < min_deadline) {
                min_deadline = abs_deadline[i];
                selected = i;
            }
        }
    }

    if (selected == -1) {
        printf("%dms: CPU is idle.\n", time);
    }
}

```

```

    } else {
        printf("%dms: Task %d is running.\n", time, selected + 1);
        remaining[selected]--; // execute
    }

    time++;
}

return 0;
}

```

Result:

```

earliest_deadline.c - CodeBlocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Plugins DoryBlocks Settings Help

<global>
main: int
1 #include <stdio.h>
2
3 #define MAX 10
4
5 int main() {
6     int n;
7
8     printf("Enter number of processes: ");
9     scanf("%d", &n);
10
11     int burst[MAX], deadline[MAX], period[MAX];
12     int remaining[MAX], abs_deadline[MAX];
13
14     printf("Enter Burst Time, Initial Deadline, Period:\n");
15     for (int i = 0; i < n; i++) {
16         scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);
17         remaining[i] = burst[i];
18         abs_deadline[i] = deadline[i];
19     }
20
21     printf("\nPID\tBurst\tDeadline\tPeriod\n");
22     for (int i = 0; i < n; i++) {
23         printf("%d\t%d\t%d\t%d\n", i+1, burst[i], deadline[i], period[i]);
24     }
25
26     int time = 0, end_time;
27
28     printf("\nEnter total scheduling time: ");
29     scanf("%d", &end_time);
30
31     printf("\nScheduling occurs for %d ms\n", end_time);
32
33     while (time < end_time) {
34
35         for (int i = 0; i < n; i++) {
36             if (time != 0 && time % period[i] == 0) {
37                 remaining[i] = burst[i];
38                 abs_deadline[i] = time + deadline[i];
39             }
40         }
41
42         int selected = -1;
43         int min_deadline = 9999;
44
45         for (int i = 0; i < n; i++) {
46             if (remaining[i] > 0) {
47                 if (abs_deadline[i] < min_deadline) {

```

```

warya M\IBM24CS024\ X
Enter number of processes: 3
Enter Burst Time, Initial Deadline, Period:
3 7 20
2 4 5
2 8 10

PID    Burst    Deadline    Period
1       3         7           20
2       2         4           5
3       2         8           10

Enter total scheduling time: 20

Scheduling occurs for 20 ms

0ms: Task 2 is running.
1ms: Task 2 is running.
2ms: Task 1 is running.
3ms: Task 1 is running.
4ms: Task 1 is running.
5ms: Task 3 is running.
6ms: Task 3 is running.
7ms: Task 2 is running.
8ms: Task 2 is running.
9ms: CPU is idle.
10ms: Task 2 is running.
11ms: Task 2 is running.
12ms: Task 3 is running.
13ms: Task 3 is running.
14ms: CPU is idle.

```

Code: c. Proportional scheduling

```

#include<stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct
{
    char name[5]; int tickets; }
Process;

```

```

int main() {
int n, total_tickets = 0; float total_T = 0.0;
printf("Enter the number of Processes: ");
scanf("%d", &n);
Process p[n];
srand(time(NULL));
for (int i = 0; i < n; i++)
{ printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: ");
scanf("%d", &p[i].tickets);
total_tickets += p[i].tickets;
total_T += p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;

scanf("%d",&m);
for (int i = 0; i < m; i++)
{
int winning_ticket = rand() % total_tickets + 1;
int accumulated_tickets = 0;
int winner_index;
for (int j = 0; j < n; j++)
{
accumulated_tickets += p[j].tickets;
if (winning_ticket <= accumulated_tickets)
{
winner_index = j; break;
}
}
printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
p[winner_index].name);

```

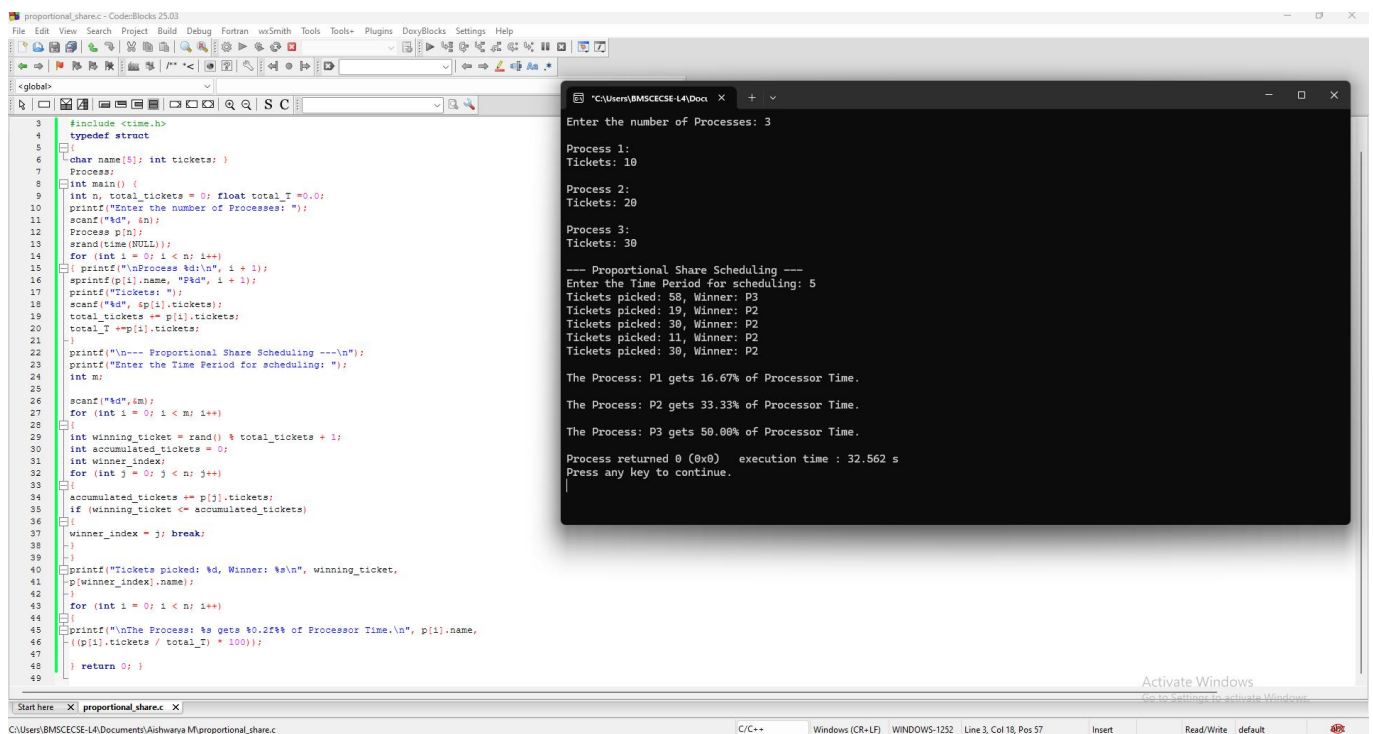
```

}
for (int i = 0; i < n; i++)
{
printf("\nThe Process: %s gets %.2f%% of Processor Time.\n", p[i].name,
((p[i].tickets / total_T) * 100));

}
return 0;
}

```

Result:



The screenshot shows a C++ IDE with the following code in `proportional_share.c`:

```

1 #include <time.h>
2
3 typedef struct
4 {
5     char name[8]; int tickets;
6 } Process;
7
8 int main() {
9     int n, total_tickets = 0; float total_T = 0.0;
10    printf("Enter the number of Processes: ");
11    scanf("%d", &n);
12    Process p[n];
13    srand(time(NULL));
14    for (int i = 0; i < n; i++)
15    { printf("\nProcess %d:\n", i + 1);
16      sprintf(p[i].name, "P%d", i + 1);
17      printf("Tickets: ");
18      scanf("%d", &p[i].tickets);
19      total_tickets += p[i].tickets;
20      total_T += p[i].tickets;
21    }
22    printf("\n--- Proportional Share Scheduling ---\n");
23    printf("Enter the Time Period for scheduling: ");
24    int m;
25
26    scanf("%d", &m);
27    for (int i = 0; i < m; i++)
28    {
29        int winning_ticket = rand() % total_tickets + 1;
30        int accumulated_tickets = 0;
31        int winner_index;
32        for (int j = 0; j < n; j++)
33        {
34            accumulated_tickets += p[j].tickets;
35            if (winning_ticket <= accumulated_tickets)
36            {
37                winner_index = j; break;
38            }
39        }
40        printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
41              p[winner_index].name);
42    }
43    for (int i = 0; i < n; i++)
44    {
45        printf("\nThe Process: %s gets %.2f%% of Processor Time.\n", p[i].name,
46              ((p[i].tickets / total_T) * 100));
47    }
48    return 0;
49 }

```

The terminal output shows the following results:

```

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 58, Winner: P3
Tickets picked: 19, Winner: P2
Tickets picked: 30, Winner: P2
Tickets picked: 11, Winner: P2
Tickets picked: 30, Winner: P2

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.

Process returned 0 (0x0)   execution time : 32.562 s
Press any key to continue.

```

Program 4)

Question: Write a C program to simulate:

- a) Producer-consumer problem using semaphores
- b) Dining-Philosopher's problem

Code :a. Producer-consumer problem using semaphores

```
#include <stdio.h>
#include <stdlib.h>

int *buffer;
int SIZE;
int in = 0, out = 0;
int mutex = 1;
int full = 0;
int empty;

void wait(int *s)
{
    (*s)--;
}

void signal(int *s)
{
    (*s)++;
}

void display()
{
    printf("\nBuffer: [ ");
    int i = out;
    for (int c = 0; c < full; c++)
    {
        printf("%d ", buffer[i]);
        i = (i + 1) % SIZE;
    }
    printf("]\n");
}

void producer()
{
    int item;

    printf("\n\nProducer tries to enter\n");
```

```

wait(&mutex);
printf("wait(mutex) -> mutex = %d\n", mutex);

if (mutex < 0)
{
    printf("Producer BLOCKED\n");
    signal(&mutex);
    return;
}

if (empty == 0)
{
    printf("Buffer FULL, cannot produce\n");
    signal(&mutex);
    return;
}

wait(&empty);

printf("\nProducer ENTERED critical section\n");

printf("\nEnter item: ");
scanf("%d", &item);

buffer[in] = item;
printf("\nProduced %d at position %d\n", item, in);

in = (in + 1) % SIZE;

printf("\nProducer leaving critical section\n");

signal(&mutex);
signal(&full);

printf("signal(mutex) -> mutex = %d\n", mutex);
}

void consumer()
{
    int item;

    printf("\n\nConsumer tries to enter\n");

    wait(&mutex);
    printf("wait(mutex) -> mutex = %d\n", mutex);

    if (mutex < 0)

```



```

{
    printf("Consumer BLOCKED\n");
    signal(&mutex);
    return;
}

if (full == 0)
{
    printf("Buffer EMPTY, cannot consume\n");
    signal(&mutex);
    return;
}

wait(&full);

printf("\nConsumer ENTERED critical section\n");

item = buffer[out];
printf("\nConsumed %d from position %d\n", item, out);

out = (out + 1) % SIZE;

printf("\nConsumer leaving critical section\n");

signal(&mutex);
signal(&empty);

printf("signal(mutex) -> mutex = %d\n", mutex);
}

void both()
{
    int item;

    printf("\n\nProducer and Consumer arrive together\n");

    printf("\nProducer tries to enter\n");
    wait(&mutex);
    printf("wait(mutex) -> mutex = %d\n", mutex);

    printf("\nProducer ENTERED critical section\n");

    printf("\nConsumer tries to enter at same time\n");
    wait(&mutex);
    printf("wait(mutex) -> mutex = %d\n", mutex);

    if (mutex < 0)

```

```

{
    printf("Consumer BLOCKED (cannot enter critical section)\n");
}

if (empty == 0)
{
    printf("\nBuffer FULL, Producer cannot proceed\n");
}
else
{
    wait(&empty);

    printf("\nEnter item: ");
    scanf("%d", &item);

    buffer[in] = item;
    printf("\nProduced %d at position %d\n", item, in);

    in = (in + 1) % SIZE;
}

printf("\nProducer leaving critical section\n");

signal(&mutex);
printf("signal(mutex) -> mutex = %d\n", mutex);

signal(&full);

printf("\nConsumer wakes up and enters critical section\n");

if (full == 0)
{
    printf("Buffer EMPTY, cannot consume\n");
    signal(&mutex);
    return;
}

wait(&full);

item = buffer[out];
printf("\nConsumed %d from position %d\n", item, out);

out = (out + 1) % SIZE;

printf("\nConsumer leaving critical section\n");

signal(&mutex);

```

```

    signal(&empty);

    printf("signal(mutex) -> mutex = %d\n", mutex);
}

int main()
{
    int choice;

    printf("Enter buffer size: ");
    scanf("%d", &SIZE);

    buffer = (int *)malloc(SIZE * sizeof(int));
    empty = SIZE;

    while (1)
    {
        printf("\n\n-----\n");
        printf("1. Produce\n2. Consume\n3. Both (simulate preemption)\n4. Exit\n");
        printf("-----\n");

        printf("\nEnter choice: ");
        scanf("%d", &choice);

        if (choice == 1)
        {
            producer();
            display();
        }
        else if (choice == 2)
        {
            consumer();
            display();
        }
        else if (choice == 3)
        {
            both();
            display();
        }
        else if (choice == 4)
        {
            free(buffer);
            return 0;
        }
        else
        {
            printf("\nInvalid choice\n");

```

```

}
}
}

```

Result:

```

bounded_buffer.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings
Help
<global>
Start here X bounded_buffer.c X
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int *buffer;
5 int SIZE;
6 int in = 0, out = 0;
7 int mutex = 1;
8 int full = 0;
9 int empty;
10
11 void wait(int *s)
12 {
13     (*s)--;
14 }
15
16 void signal(int *s)
17 {
18     (*s)++;
19 }
20
21 void display()
22 {
23     printf("\nBuffer: [ ");
24     int i = out;
25     for (int c = 0; c < full; c++)
26     {
27         printf("%d ", buffer[i]);
28         i = (i + 1) % SIZE;
29     }
30     printf("]\n");
31 }
32
33 void producer()

```

```

Enter buffer size: 5
-----
1. Produce
2. Consume
3. Both (simulate preemption)
4. Exit
-----
Enter choice: 3

Producer and Consumer arrive together

Producer tries to enter
wait(mutex) -> mutex = 0

Producer ENTERED critical section

Consumer tries to enter at same time
wait(mutex) -> mutex = -1
Consumer BLOCKED (cannot enter critical section)

Enter item: 10

Produced 10 at position 0

Producer leaving critical section
signal(mutex) -> mutex = 0

Consumer wakes up and enters critical section

Consumed 10 from position 0

Consumer leaving critical section
signal(mutex) -> mutex = 1

Buffer: [ ]
-----
1. Produce
2. Consume
3. Both (simulate preemption)
4. Exit

```

Code: b. Dining-Philosopher's problem

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

```

```

#define N 5

```

```

pthread_mutex_t forks[N];
pthread_t philosophers[N];

```

```

void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;

```

```

while (1)
{
    printf("Philosopher %d is THINKING\n", id);
    sleep(1);

    if (id % 2 == 0)
    {
        // Even → left then right
        pthread_mutex_lock(&forks[left]);
        printf("P%d picked LEFT fork %d\n", id, left);

        pthread_mutex_lock(&forks[right]);
        printf("P%d picked RIGHT fork %d\n", id, right);
    }
    else
    {
        // Odd → right then left
        pthread_mutex_lock(&forks[right]);
        printf("P%d picked RIGHT fork %d\n", id, right);

        pthread_mutex_lock(&forks[left]);
        printf("P%d picked LEFT fork %d\n", id, left);
    }

    printf("Philosopher %d is EATING\n", id);
    sleep(2);

    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);

    printf("P%d released forks %d and %d\n\n", id, left, right);
}

return NULL;
}

int main()
{
    int ids[N];

    for (int i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
}

```

```

for (int i = 0; i < N; i++)
{
    pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
}

for (int i = 0; i < N; i++)
{
    pthread_join(philosophers[i], NULL);
}

for (int i = 0; i < N; i++)
{
    pthread_mutex_destroy(&forks[i]);
}

return 0;
}

```

Result:

The screenshot displays the Code::Blocks IDE with the following components:

- Source Code (Left Pane):** Shows the implementation of the Dining Philosopher problem. It includes headers for `<stdio.h>`, `<pthread.h>`, and `<unistd.h>`. A macro `N` is defined as 5. The `philosopher` function takes a thread ID and implements a loop where each philosopher picks up two forks (left and right), eats, and then releases them. The code uses `pthread_mutex_t` to manage the forks and `sleep(1)` to simulate thinking time.
- Output (Right Pane):** Shows the runtime output of the program. It details the sequence of events for five philosophers (P0-P4), including their thinking, picking up forks, eating, and releasing forks. The output demonstrates a valid execution where no two philosophers hold the same fork simultaneously.
- Build Log (Bottom Pane):** Shows the compilation process. It indicates that the build was successful with no errors or warnings.

Program 5)

Question: Write a C program to simulate

- a) Banker's algorithm for the purpose of deadlock avoidance.
- b) Deadlock Detection

Code :a. Banker's algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>

int main()
{
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("Enter Maximum Demand Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            scanf("%d", &max[i][j]);
        }
    }

    printf("Enter Available Resources:\n");
    for(i = 0; i < m; i++)
    {
        scanf("%d", &avail[i]);
    }

    for(i = 0; i < n; i++)
    {
```

```

    for(j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int finish[n], safeSeq[n];
for(i = 0; i < n; i++)
{
    finish[i] = 0;
}

int count = 0;

while(count < n)
{
    int found = 0;
    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int flag = 1;
            for(j = 0; j < m; j++)
            {
                if(need[i][j] > avail[j])
                {
                    flag = 0;
                    break;
                }
            }

            if(flag)
            {
                for(k = 0; k < m; k++)
                {
                    avail[k] += alloc[i][k];
                }

                safeSeq[count] = i;
                count++;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)

```



```

    {
        printf("\nSystem is not in safe state.\n");
        return 0;
    }
}

printf("\nSystem is in a safe state.\n");
printf("Safe sequence is: ");

for(i = 0; i < n; i++)
{
    printf("P%d", safeSeq[i]);

    if(i != n - 1)
        printf(" -> ");
}

printf("\n");

return 0;
}

```

Result:

The screenshot displays a C++ IDE with the source code of a program named `banker.c` and its execution output in a terminal window.

Source Code (banker.c):

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int n, m, i, j, k;
6
7      printf("Enter number of processes: ");
8      scanf("%d", &n);
9
10     printf("Enter number of resources: ");
11     scanf("%d", &m);
12
13     int alloc[n][m], max[n][m], need[n][m];
14     int avail[m];
15
16     printf("Enter Allocation Matrix:\n");
17     for(i = 0; i < n; i++)
18     {
19         for(j = 0; j < m; j++)
20         {
21             scanf("%d", &alloc[i][j]);
22         }
23     }
24
25     printf("Enter Maximum Demand Matrix:\n");
26     for(i = 0; i < n; i++)
27     {
28         for(j = 0; j < m; j++)
29         {
30             scanf("%d", &max[i][j]);
31         }
32     }
33 }

```

Execution Output:

```

Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2

Process returned 0 (0x0)   execution time : 52.936 s
Press any key to continue.

```

The IDE interface includes a menu bar (File, Edit, View, Search, Project, Build, Debug, Fortran, wxSmith, Tools, Tools+, Plugins, DoxyBlocks, Settings), a toolbar, and a status bar at the bottom showing the current file path and line/column information.

Code: b. Deadlock Detection

```
#include <stdio.h>

int main()
{
    int n, m, i, j, k;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], request[n][m], avail[m];
    int finish[n], safeSeq[n];
    printf("Enter the allocation matrix:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("Enter the request matrix:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++)
        {
            scanf("%d", &request[i][j]);
        }
    }

    printf("Enter the available resources: ");
    for(i = 0; i < m; i++)
    {
        scanf("%d", &avail[i]);
    }
    for(i = 0; i < n; i++)
    {
        finish[i] = 0;
    }

    int count = 0;
```

```

while(count < n)
{
    int found = 0;
    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int flag = 1;

            for(j = 0; j < m; j++)
            {
                if(request[i][j] > avail[j])
                {
                    flag = 0;
                    break;
                }
            }

            if(flag)
            {
                for(k = 0; k < m; k++)
                {
                    avail[k] += alloc[i][k];
                }
                safeSeq[count] = i;
                count++;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
    {
        break;
    }
}

if(count == n)
{
    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d", safeSeq[i]);
    }
}

```

```

        if(i != n - 1)
            printf(" ");
    }
    printf("\n");
}
else
{
    printf("\nDeadlock detected.\nProcesses in deadlock are: ");

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            printf("P%d ", i);
        }
    }

    printf("\n");
}
return 0;
}

```

Result:

The screenshot displays a code editor window titled 'deadlock_detection.c - Code::Blocks 25.03' on the left and a terminal window on the right. The code in the editor is a C program for deadlock detection using the Banker's algorithm. It prompts the user for the number of processes (5), resources (3), and allocation and request matrices. The terminal output shows the input matrices and the program's conclusion that the system is in a safe state with a safe sequence of P0, P2, P3, P4, P1. The execution time is 94.088 seconds.

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int n, m, i, j, k;
6
7      printf("Enter the number of processes: ");
8      scanf("%d", &n);
9
10     printf("Enter the number of resources: ");
11     scanf("%d", &m);
12
13     int alloc[n][m], request[n][m], avail[m];
14     int finish[n], safeSeq[n];
15
16     printf("Enter the allocation matrix:\n");
17     for(i = 0; i < n; i++)
18     {
19         printf("Process %d: ", i);
20         for(j = 0; j < m; j++)
21         {
22             scanf("%d", &alloc[i][j]);
23         }
24     }
25
26     printf("Enter the request matrix:\n");
27     for(i = 0; i < n; i++)
28     {
29         printf("Process %d: ", i);
30         for(j = 0; j < m; j++)
31         {
32             scanf("%d", &request[i][j]);
33         }
34     }
35
36     // Banker's algorithm logic would go here
37 }

```

```

Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0

System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1

Process returned 0 (0x0)   execution time : 94.088 s
Press any key to continue.

```

Program 6)

Question: Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst-fit b) Best-fit c) First-fit

Code:

```
#include <stdio.h>

void firstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                allocation[i] = j;

                blockSize[j] = 0;

                break;
            }
        }
    }

    printf("\n--- First Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for(int i = 0; i < n; i++)
    {
        printf("%d\t%d\t", i + 1, processSize[i]);

        if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n)
{

```

```

int allocation[n];

for(int i = 0; i < n; i++)
    allocation[i] = -1;

for(int i = 0; i < n; i++)
{
    int bestIdx = -1;
    for(int j = 0; j < m; j++)
    {
        if(blockSize[j] >= processSize[i])
        {
            if(bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
            {
                bestIdx = j;
            }
        }
    }

    if(bestIdx != -1)
    {
        allocation[i] = bestIdx;
        blockSize[bestIdx] = 0;
    }
}

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for(int i = 0; i < n; i++)
{
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);
    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++)
    {

```

```

    int worstIdx = -1;
    for(int j = 0; j < m; j++)
    {
        if(blockSize[j] >= processSize[i])
        {
            if(worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
            {
                worstIdx = j;
            }
        }
    }

    if(worstIdx != -1)
    {
        allocation[i] = worstIdx;
        blockSize[worstIdx] = 0;
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for(int i = 0; i < n; i++)
{
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);

    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

int main()
{
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blockSize[m], block1[m], block2[m], block3[m];

    printf("Enter sizes of %d memory blocks:\n", m);

    for(int i = 0; i < m; i++)
    {
        scanf("%d", &blockSize[i]);
    }
}

```

```

    block1[i] = blockSize[i];
    block2[i] = blockSize[i];
    block3[i] = blockSize[i];
}

printf("\nEnter number of processes: ");
scanf("%d", &n);

int processSize[n];

printf("Enter sizes of %d processes:\n", n);

for(int i = 0; i < n; i++)
{
    scanf("%d", &processSize[i]);
}

firstFit(block1, m, processSize, n);
bestFit(block2, m, processSize, n);
worstFit(block3, m, processSize, n);

return 0;
}

```

Result:

The screenshot displays a C++ IDE with the source code for memory allocation algorithms and a terminal window showing the program's execution.

Source Code:

```

1 #include <stdio.h>
2
3 void firstFit(int blockSize[], int m, int processSize[], int n)
4 {
5     int allocation[n];
6
7     for(int i = 0; i < n; i++)
8         allocation[i] = -1;
9
10    for(int i = 0; i < n; i++)
11    {
12        for(int j = 0; j < m; j++)
13        {
14            if(blockSize[j] >= processSize[i])
15            {
16                allocation[i] = j;
17                blockSize[j] = 0;
18                break;
19            }
20        }
21    }
22
23    printf("\n--- First Fit ---\n");
24    printf("Process No.\tProcess Size\tBlock No.\n");
25
26    for(int i = 0; i < n; i++)
27    {
28        printf("%d\t%d\t", i + 1, processSize[i]);
29
30        if(allocation[i] != -1)
31            printf("%d\n", allocation[i] + 1);
32        else
33            printf("Not Allocated\n");
34    }
35
36    void bestFit(int blockSize[], int m, int processSize[], int n)
37    {
38        int allocation[n];
39
40        for(int i = 0; i < n; i++)
41            allocation[i] = -1;
42
43        for(int i = 0; i < n; i++)
44        {
45
46

```

Execution Output:

```

400 700 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 512 312 526

--- First Fit ---
Process No. Process Size Block No.
1          212         1
2          512         2
3          312         5
4          526        Not Allocated

--- Best Fit ---
Process No. Process Size Block No.
1          212         4
2          512         5
3          312         1
4          526         2

--- Worst Fit ---
Process No. Process Size Block No.
1          212         2
2          512         5
3          312         1
4          526        Not Allocated

Process returned 0 (0x0)   execution time : 29.349 s
Press any key to continue.

```


Program 7)

Question: Write a C program to simulate page replacement algorithms

a) FIFO b) LRU c) Optimal

Code:

```
#include <stdio.h>

#define MAX 100

int isPresent(int frames[], int num_frames, int page)
{
    for (int i = 0; i < num_frames; i++)
    {
        if (frames[i] == page)
            return 1;
    }

    return 0;
}

void printFrames(int frames[], int num_frames)
{
    printf("[");
    for (int i = 0; i < num_frames; i++)
    {
        if (frames[i] != -1)
            printf("%d ", frames[i]);
    }

    printf("]\n");
}

void FIFO(int pages[], int n, int num_frames)
{
    int frames[MAX];
    int page_faults = 0;
    int next = 0;

    for (int i = 0; i < num_frames; i++)
        frames[i] = -1;

    printf("\n--- FIFO Page Replacement ---\n");
```

```

for (int i = 0; i < n; i++)
{
    int page = pages[i];

    if (!isPresent(frames, num_frames, page))
    {
        frames[next] = page;
        next = (next + 1) % num_frames;
        page_faults++;
    }

    printf("Page %d -> ", page);
    printFrames(frames, num_frames);
}

printf("Total Page Faults (FIFO): %d\n", page_faults);
}

```

```

void LRU(int pages[], int n, int num_frames)
{
    int frames[MAX];
    int lastUsed[MAX];
    int page_faults = 0;

    for (int i = 0; i < num_frames; i++)
    {
        frames[i] = -1;
        lastUsed[i] = -1;
    }

    printf("\n--- LRU Page Replacement ---\n");

    for (int i = 0; i < n; i++)
    {
        int page = pages[i];
        int found = 0;

        for (int j = 0; j < num_frames; j++)
        {
            if (frames[j] == page)
            {
                found = 1;
                lastUsed[j] = i;
                break;
            }
        }
    }
}

```

```

    }
}

if (!found)
{
    int pos = -1;

    for (int j = 0; j < num_frames; j++)
    {
        if (frames[j] == -1)
        {
            pos = j;
            break;
        }
    }

    if (pos == -1)
    {
        int lruIndex = 0;

        for (int j = 1; j < num_frames; j++)
        {
            if (lastUsed[j] < lastUsed[lruIndex])
                lruIndex = j;
        }

        pos = lruIndex;
    }

    frames[pos] = page;
    lastUsed[pos] = i;
    page_faults++;
}

printf("Page %d -> ", page);
printFrames(frames, num_frames);
}

printf("Total Page Faults (LRU): %d\n", page_faults);
}

void Optimal(int pages[], int n, int num_frames)
{
    int frames[MAX];
    int page_faults = 0;

```

```

// Initialize frames
for (int i = 0; i < num_frames; i++)
    frames[i] = -1;

printf("\n--- Optimal Page Replacement ---\n");

for (int i = 0; i < n; i++)
{
    int page = pages[i];

    if (!isPresent(frames, num_frames, page))
    {
        int pos = -1;

        for (int j = 0; j < num_frames; j++)
        {
            if (frames[j] == -1)
            {
                pos = j;
                break;
            }
        }

        if (pos == -1)
        {
            int farthest = i;
            int replaceIndex = -1;

            for (int j = 0; j < num_frames; j++)
            {
                int k;

                for (k = i + 1; k < n; k++)
                {
                    if (frames[j] == pages[k])
                        break;
                }

                if (k == n)
                {
                    replaceIndex = j;
                    break;
                }
            }
        }
    }
}

```

```

        }

        if (k > farthest)
        {
            farthest = k;
            replaceIndex = j;
        }
    }

    pos = replaceIndex;
}

frames[pos] = page;
page_faults++;
}

printf("Page %d -> ", page);
printFrames(frames, num_frames);
}

printf("Total Page Faults (Optimal): %d\n", page_faults);
}

int main()
{
    int pages[MAX], n, num_frames;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &num_frames);

    FIFO(pages, n, num_frames);
    LRU(pages, n, num_frames);
    Optimal(pages, n, num_frames);

    return 0;
}

```

Result:

```
page.cpp: CodeBlocks 25.03
Be Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DayBlocks Settings Help

rglobab:
#include <stdio.h>
#define MAX 100
int isPresent(int frames[], int num_frames, int page)
{
    for (int i = 0; i < num_frames; i++)
    {
        if (frames[i] == page)
            return 1;
    }
    return 0;
}
void printFrames(int frames[], int num_frames)
{
    printf("[");
    for (int i = 0; i < num_frames; i++)
    {
        if (frames[i] != -1)
            printf("%d ", frames[i]);
    }
    printf("]\n");
}
void FIFO(int pages[], int n, int num_frames)
{
    int frames[MAX];
    int page_faults = 0;
    int next = 0;
    for (int i = 0; i < num_frames; i++)
        frames[i] = -1;
    printf("\n--- FIFO Page Replacement ---\n");
}
// ... (FIFO logic continues) ...
// ... (LRU logic continues) ...
// ... (Optimal logic continues) ...

Enter number of pages: 21
Enter page reference string:
5 0 1 0 2 3 0 2 4 3 3 2 0 2 1 2 7 0 1 1 0
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 5 -> [5 ]
Page 0 -> [5 0 ]
Page 1 -> [5 0 1 ]
Page 0 -> [5 0 1 ]
Page 2 -> [2 0 1 ]
Page 3 -> [2 3 1 ]
Page 0 -> [2 3 0 ]
Page 2 -> [2 3 0 ]
Page 4 -> [4 3 0 ]
Page 3 -> [4 3 0 ]
Page 3 -> [4 3 0 ]
Page 2 -> [4 2 0 ]
Page 0 -> [4 2 0 ]
Page 2 -> [4 2 0 ]
Page 1 -> [4 2 1 ]
Page 2 -> [4 2 1 ]
Page 7 -> [7 2 1 ]
Page 0 -> [7 0 1 ]
Page 0 -> [7 0 1 ]
Page 1 -> [7 0 1 ]
Page 1 -> [7 0 1 ]
Page 0 -> [7 0 1 ]
Total Page Faults (FIFO): 11

--- LRU Page Replacement ---
Page 5 -> [5 ]
Page 0 -> [5 0 ]
Page 1 -> [5 0 1 ]
Page 0 -> [5 0 1 ]
Page 2 -> [2 0 1 ]
Page 3 -> [2 0 3 ]
Page 0 -> [2 0 3 ]
Page 2 -> [2 0 3 ]
Page 4 -> [2 0 4 ]
Page 3 -> [2 3 4 ]
Page 3 -> [2 3 4 ]
Page 2 -> [2 3 4 ]
Page 0 -> [2 3 0 ]
Page 1 -> [2 1 0 ]
Page 2 -> [2 1 0 ]
Page 7 -> [2 1 7 ]
Page 0 -> [2 0 7 ]
Page 0 -> [2 0 7 ]
Page 1 -> [1 0 7 ]
Page 1 -> [1 0 7 ]
Page 0 -> [1 0 7 ]
Total Page Faults (LRU): 12

--- Optimal Page Replacement ---
Page 5 -> [5 ]
Page 0 -> [5 0 ]
Page 1 -> [5 0 1 ]
Page 0 -> [5 0 1 ]
Page 2 -> [2 0 1 ]
Page 3 -> [2 0 3 ]
Page 0 -> [2 0 3 ]
Page 2 -> [2 0 3 ]
Page 4 -> [2 4 3 ]
Page 3 -> [2 4 3 ]
Page 3 -> [2 4 3 ]
Page 2 -> [2 4 3 ]
Page 0 -> [2 0 3 ]
Page 1 -> [2 0 1 ]
Page 2 -> [2 0 1 ]
Page 7 -> [7 0 1 ]
Page 0 -> [7 0 1 ]
Page 1 -> [7 0 1 ]
Page 1 -> [7 0 1 ]
Page 0 -> [7 0 1 ]
Total Page Faults (Optimal): 9

Process returned 0 (0x0)   execution time : 30.672 s
Press any key to continue.
```